Logic Gates

Contemporary Digital Electronic Systems — Chapter 4

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Basic Logic Gates
3. Derived Logic Gates
4. Boolean Theorems
5. Gate ICs
6. Measuring Performance
7. Alternative Symbols
8. Summary

Why This Chapter Matters

Logic gates are the atoms of digital hardware. Every combinational circuit later in the module (adders, decoders, multiplexers, controllers) is wired from a handful of gate types. We need their symbols, truth tables and real electrical behaviour first.

After this chapter, you can:

1. Draw the symbol and truth table of AND, OR, NOT, NAND, NOR, XOR, XNOR.
2. Explain why NAND and NOR are universal; build a NOT and an AND from NAND.
3. Interpret propagation delay and power dissipation.
4. Compare CMOS and TTL power.
5. Recognise a tri-state buffer and where it belongs.

© Munim Zabidi

An Opening Question

If you had a single gate type to build every digital circuit, which would you pick?

The answer may surprise you. We return to it with NAND.

Table of Contents

1 Why This Chapter

2 **Basic Logic Gates**

3 Derived Logic Gates

4 Boolean Theorems

5 Gate ICs

6 Measuring Performance

7 Alternative Symbols

8 Summary

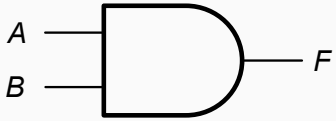
Gates, Boolean Algebra and Truth Tables



- A gate has one or more inputs and one output set by its function.
- Boolean algebra: variables take values 0 or 1; complement written $\overline{A}$ or A'.
- A truth table lists output for every input combination; inputs left, output right.
- A canonical truth table for n inputs has 2^n rows.

© Muním Zabidi

AND Gate



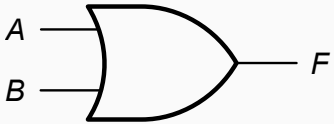
$$F = A \cdot B = AB$$

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

HIGH only when all inputs are HIGH. Like switches in series: the lamp lights only if both are closed.

© Munim Zabidi

OR Gate



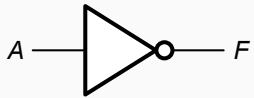
$$F = A + B$$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

HIGH when at least one input is HIGH.
Like switches in parallel.

© Muním Zabidi

NOT Gate (Inverter)



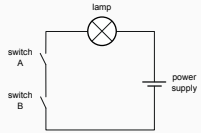
$F = \overline{A}$

A	F
0	1
1	0

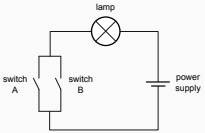
One input. Output is the opposite level. The bubble at the output denotes inversion.

© Muním Zabidi

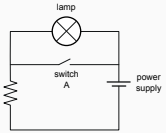
Switch Models of the Basic Gates



AND: series



OR: parallel

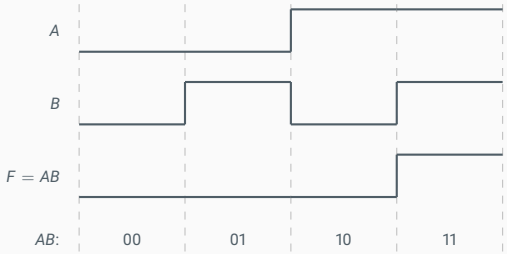


NOT: switch across the lamp

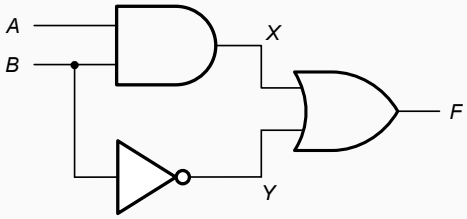
- Open switch = logic 0; closed switch = logic 1; lamp = output.
- This is why digital circuits are called switching circuits.

© Munim Zabidi

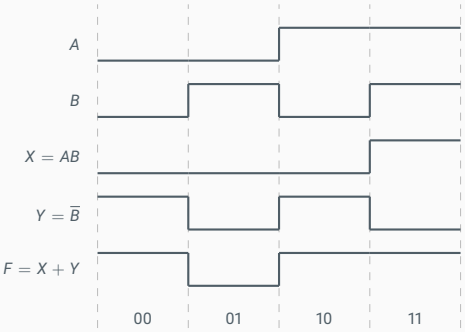
Timing Diagram of a Gate



- A truth table has no notion of time. A timing diagram plots levels against time.
- Each interval matches one truth-table row.
- Inputs on top, outputs below.



$X = AB, \quad Y = \overline{B}, \quad F = X + Y$



F is LOW only when X and Y are both LOW: $A = 0,\ B = 1$. For $AB = 10$: $X = 0$, $Y = 1$, so $F = 1$.

© Munim Zabidi

More Than Two Inputs



- NOT has exactly one input; AND and OR can have many.
- AND is 1 only if every input is 1; OR is 1 if any input is 1.
- A 3-input gate has $2^3 = 8$ truth-table rows (000 to 111). The 3-input AND is 1 only on row 111.
- *n*-input NAND is 0 only when all inputs are 1; *n*-input NOR is 1 only when all inputs are 0.
- XOR is the exception: it is defined for two inputs. Parity over several bits is built by cascading two-input XOR gates (Chapter 9).

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Basic Logic Gates

3 **Derived Logic Gates**

4 Boolean Theorems

5 Gate ICs

6 Measuring Performance

7 Alternative Symbols

8 Summary

NAND Gate



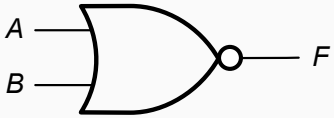
$$F = \overline{AB}$$

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

NOT-AND: LOW only when all inputs are HIGH. Equivalent to an AND followed by a NOT.

© Munim Zabidi

NOR Gate



$$F = \overline{A + B}$$

A	B	F
0	0	1
0	1	0
1	0	0
1	1	0

NOT-OR: HIGH only when all inputs are LOW. Equivalent to an OR followed by a NOT.

NAND Is Universal

NOT from NAND

Tie both inputs together:
$\overline{A \cdot A} = \overline{A}$

AND from NAND

NAND followed by a NAND wired as NOT:
$\overline{\overline{AB}} = AB$

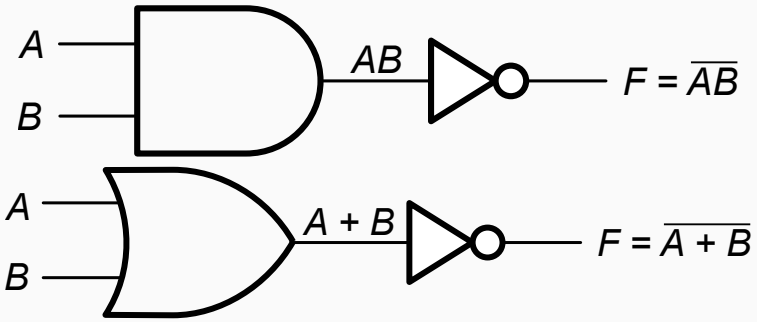
Universal gates

Any Boolean function can be built from NAND gates alone, or from NOR gates alone.

Back to the opening question: pick NAND (or NOR). AND alone or OR alone cannot invert, so they cannot build most functions. One gate type, one part number, every circuit (Section 7.7 shows a circuit rebuilt to fit one NAND package).

© Muním Zabidi

Equivalent Forms of NAND and NOR

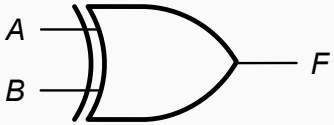


NAND = AND + NOT

NOR = OR + NOT

© Muním Zabidi

XOR Gate

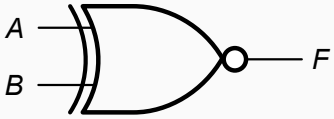


$$F = \overline{A}B + A\overline{B} = A \oplus B$$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

HIGH when the inputs differ. Used in adders and in circuits that detect differences between binary values.

XNOR Gate



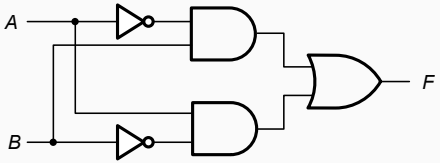
$$F = AB + \overline{A}\,\overline{B} = A \odot B$$

A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

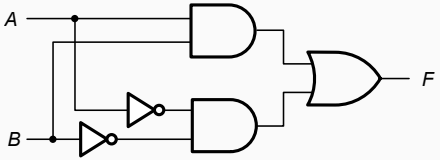
HIGH when the inputs are equal. Opposite of XOR. Useful for comparing bits: output is 1 when they match.

© Muním Zabidi

XOR and XNOR from Basic Gates



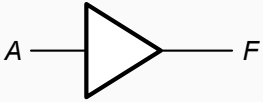
XOR from AND, OR, NOT



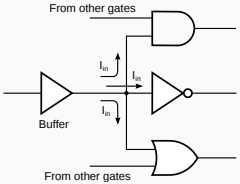
XNOR = XOR followed by NOT

© Muním Zabidi

Buffers

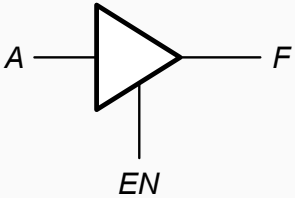


- One input, one output; $F = A$. The logic value is unchanged.
- It supplies more current than the source, e.g. to drive an LED array, and isolates stages.
- Larger loads such as relays and solenoids need a proper driver circuit.



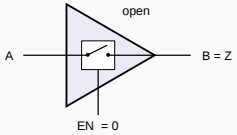
© Muním Zabidi

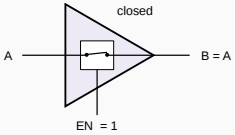
Tri-State Buffer



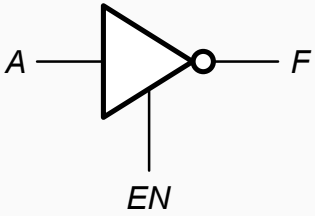
EN	A	F
1	0	0
1	1	1
0	X	Hi-Z

- An enable input controls the output.
- Enabled: output follows input. Disabled: high impedance (Hi-Z), electrically disconnected.
- Several devices can share one line (a bus) if only one drives it at a time.
- Variants: inverting, and active-low enable $\overline{EN}$.



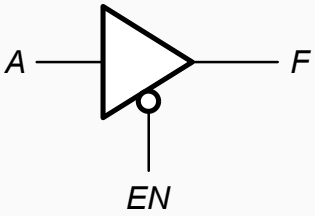


Buffer Variations



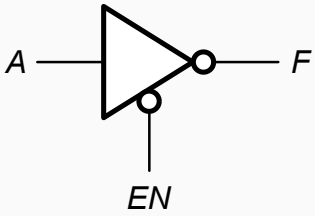
Inverting, active-high *EN*

EN	*A*	*F*
1	0	1
1	1	0
0	X	Hi-Z



Non-inverting, active-low $\overline{EN}$

$\overline{EN}$	*A*	*F*
0	0	0
0	1	1
1	X	Hi-Z



Inverting, active-low $\overline{EN}$

$\overline{EN}$	*A*	*F*
0	0	1
0	1	0
1	X	Hi-Z

Each gives signal isolation and controlled drive in a different configuration.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Basic Logic Gates

3 Derived Logic Gates

4 **Boolean Theorems**

5 Gate ICs

6 Measuring Performance

7 Alternative Symbols

8 Summary

- George Boole (1854) based it on AND, OR, NOT. Claude Shannon (1930s) applied it to two-state switching circuits.
- Theorems simplify expressions, cutting gates, cost and failure points.

$$
\begin{array}{cc}
0 \cdot 0 = 0 & 1 + 1 = 1 \\
1 \cdot 1 = 1 & 0 + 0 = 0 \\
0 \cdot 1 = 1 \cdot 0 = 0 & 0 + 1 = 1 + 0 = 1 \\
\overline{1} = 0 & \overline{0} = 1
\end{array}
$$

Here $+$ is OR, $\cdot$ is AND, and the overbar is NOT.

© Munim Zabidi

	Theorem	AND form	OR form
T1	Annulment	$A \cdot 0 = 0$	$A + 1 = 1$
T2	Identity	$A \cdot 1 = A$	$A + 0 = A$
T3	Idempotency	$A \cdot A = A$	$A + A = A$
T4	Complements	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
T5	Involution	$\overline{\overline{A}} = A$	

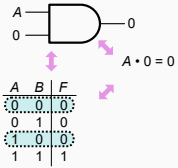
- Duality: swap AND with OR, and 0 with 1, to get the dual theorem.

Example: $F = A \cdot A \cdot \overline{A} + 1$

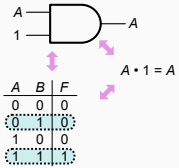
T3: $A \cdot A = A$. T4: $A \cdot \overline{A} = 0$. So $F = 0 + 1 = 1$. (Directly by T1: anything $+ 1 = 1$.)

© Muním Zabidi

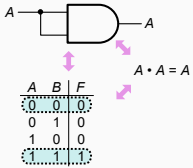
Theorems as Circuits: T1–T3



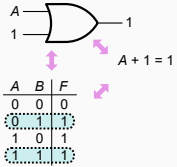
$A \cdot 0 = 0$



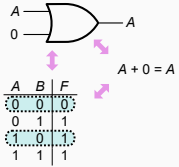
$A \cdot 1 = A$



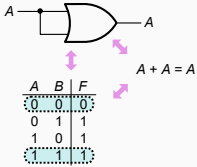
$A \cdot A = A$



$A + 1 = 1$



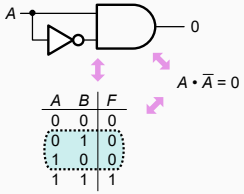
$A + 0 = A$



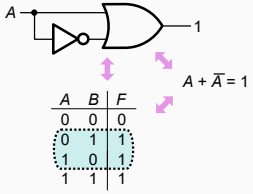
$A + A = A$

T1: one input forces the output on its own. T2: a 1 on an AND (0 on an OR) makes the gate transparent. T3: *A* can only equal itself, so the gate can be deleted.

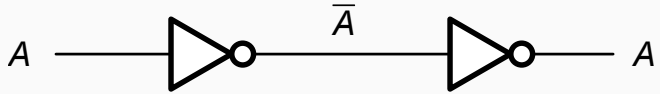
Theorems as Circuits: T4–T5



$A \cdot \overline{A} = 0$



$A + \overline{A} = 1$



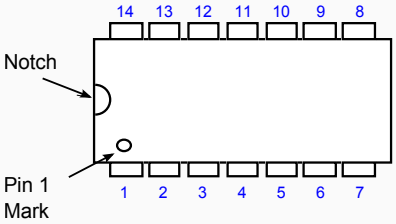
$\overline{\overline{A}} = A$

T4: A and $\overline{A}$ are never high together, and one is always high. T5: two inversions cancel, so the pair can be removed.

Table of Contents

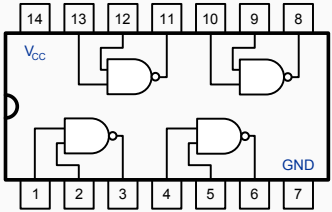
1. Why This Chapter
2. Basic Logic Gates
3. Derived Logic Gates
4. Boolean Theorems
5. **Gate ICs**
6. Measuring Performance
7. Alternative Symbols
8. Summary

- SSI chips of the 74 series hold four or six gates each: “quad” and “hex”.
- DIP suits labs and breadboards; SOT-23 and DFN fit tight spaces.
- DIP14: pins numbered counter-clockwise from pin 1, beside the notch.
- Pinouts vary: always check the datasheet.

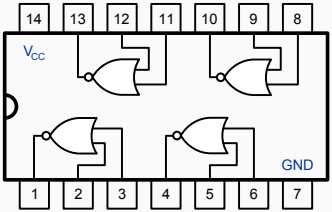


© Munim Zabidi

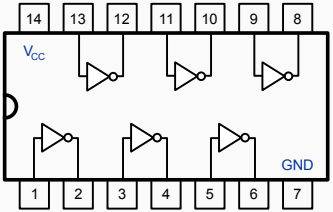
74-Series Examples



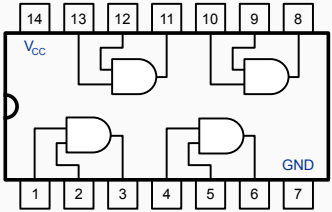
74x00 quad NAND



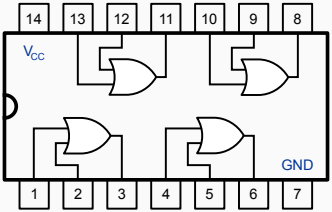
74x02 quad NOR



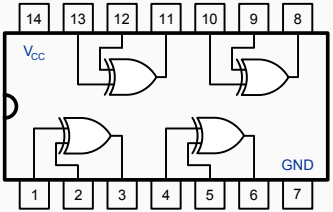
74x04 hex inverter



74x08 quad AND



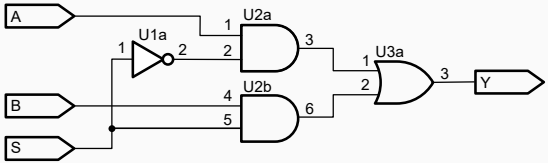
74x32 quad OR



74x86 quad XOR

© Muním Zabidi

Reading a Schematic



- U1 (74x04): inverter U1a, pins 1–2.
- U2 (74x08): AND gates U2a (pins 1, 2, 3) and U2b (pins 4, 5, 6).
- U3 (74x32): OR gate U3a, pins 1, 2, 3.
- The designator names the IC; the letter names the gate inside it.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 Basic Logic Gates

3 Derived Logic Gates

4 Boolean Theorems

5 Gate ICs

6 **Measuring Performance**

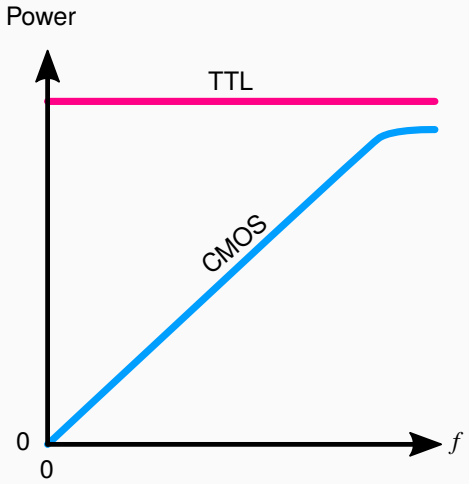
7 Alternative Symbols

8 Summary

CMOS total power = static + dynamic

$$P_{\mathrm{s}} = I_{\mathrm{dd}} V_{\mathrm{dd}} \quad \text{(leakage)}$$
$$P_{\mathrm{dyn}} = C_L V_{\mathrm{dd}}^2 f$$

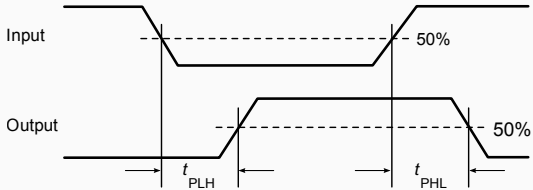
- C_L: load capacitance; V_{dd}: supply; f: switching frequency.
- CMOS idles at very low power; its power rises with f.
- TTL draws roughly constant power regardless of speed.



Propagation Delay

- A gate does not respond instantly. Propagation delay is the time from an input transition to the output transition.
- t_{PHL}: output HIGH to LOW. t_{PLH}: output LOW to HIGH.

$$t_{\mathrm{pd}} = \frac{t_{\mathrm{PHL}} + t_{\mathrm{PLH}}}{2}$$



- Typical speed order: inverters, then NAND/NOR, then AND/OR, then XOR/XNOR.
- The path with the greatest total delay is the critical path; it limits the maximum operating speed.

Table of Contents

1 Why This Chapter

2 Basic Logic Gates

3 Derived Logic Gates

4 Boolean Theorems

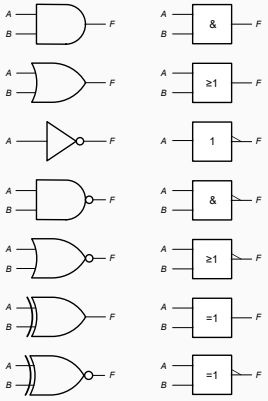
5 Gate ICs

6 Measuring Performance

7 **Alternative Symbols**

8 Summary

Conventional and Rectangular Symbols



- ANSI/IEEE Std 91-1984 rectangular symbols: `&` = AND, `1` = OR, `=1` = exactly one input high (XOR for two inputs; XOR itself is defined for two). A triangle replaces the bubble.

Table of Contents

1. Why This Chapter
2. Basic Logic Gates
3. Derived Logic Gates
4. Boolean Theorems
5. Gate ICs
6. Measuring Performance
7. Alternative Symbols
8. **Summary**

Summary

- A truth table defines a gate's function; the symbol is its graphical form.
- NAND and NOR are universal: any Boolean function from one gate type.
- Propagation delay limits speed; the critical path sets the maximum frequency.
- Dynamic CMOS power $\approx CV_{\mathrm{DD}}^2 f$ for constant switching activity.
- A tri-state buffer adds a Hi-Z state so outputs can share a bus, one driver at a time.

Looking ahead

Chapter 5 uses Boolean algebra to simplify the logic that these gates implement.